

Report

Gona Sanjana - CS23BTECH11019

April 21, 2024

1 Design Section

The Infinite Arithmetic Library is designed to provide a robust and efficient solution for performing arithmetic operations on integers and floating-point numbers of arbitrary size. The library follows an object-oriented design approach, leveraging the principles of encapsulation, abstraction, and modularity.

1.1 Class Design

The library consists of two main classes: Integer and Float, each responsible for handling arithmetic operations on their respective number types.

1.1.1 Integer Class

The Integer class encapsulates the functionality for performing arithmetic operations on integers of arbitrary size. It uses a string to represent the integer value internally, allowing for virtually unlimited precision. The class provides the following features: Constructors (default, string, and copy), Arithmetic Operations (overloaded $+$, $-$, $*$, and $/$ operators), and Helper Functions (parsing strings, removing leading/trailing zeroes, handling negative signs).

1.1.2 Float Class

The Float class is designed to handle arithmetic operations on floating-point numbers of arbitrary size. Similar to the Integer class, it uses a string to represent the floating-point value internally. The class provides the following features: Constructors (default, string, and copy), Arithmetic Operations (overloaded $+$, $-$, and $*$ operators), and Helper Functions (shared with Integer class, converting strings to integers, handling decimal points).

1.2 Design Decisions

Several design decisions were made to ensure the library's robustness, efficiency, and extensibility.

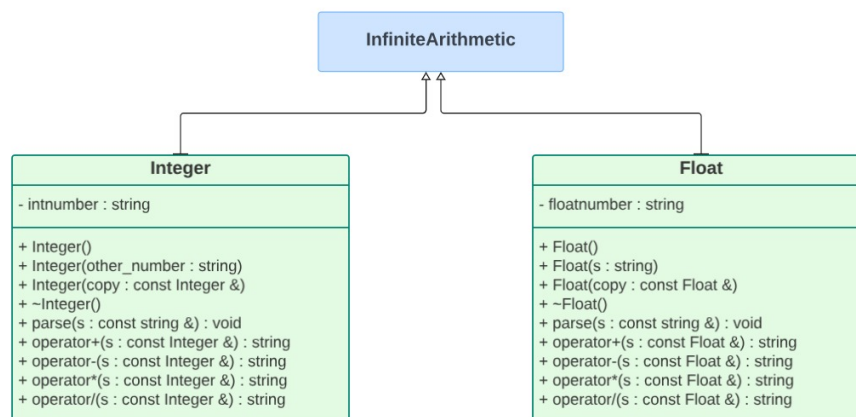


Figure 1: UML DIAGRAM

1. **String Representation:** Both **Integer** and **Float** classes use strings to represent their number types internally to accommodate arbitrary size without limitations of fixed-size data types. String manipulation is generally efficient for large numbers.
2. **Helper Functions:** Helper functions handle tasks like removing leading/-trailing zeroes, checking negative signs, and converting strings to integers for improved reusability and maintainability.
3. **Modularity:** The library is designed to be modular, with each class and function having well-defined responsibilities, facilitating code organization, testability, and future extensions.

2 READ ME

The Infinite Arithmetic Library provides a simple and convenient way to perform arithmetic operations on integers and floating-point numbers of arbitrary size. The library is implemented in C++ and can be easily integrated into your projects.

2.1 Usage

1. Include the header file **my_inf_arith.h** in your C++ source file.
2. Use the **InfiniteArithmetic** namespace.
3. Create objects of the **Integer** or **Float** class, depending on your needs.

4. Perform arithmetic operations using the overloaded operators (+, -, *, / for **Integer**, and +, -, *, / for **Float**).

2.2 Example

```
1  #include "my_inf_arith.h"
2  using namespace InfiniteArithmetic;
3  using namespace std;
4
5  int main()
6  {
7      Integer num1("1234567890");
8      Integer num2("9876543210");
9
10     cout << "Addition: " << num1 + num2 << endl;
11     cout << "Subtraction: " << num1 - num2 << endl;
12     cout << "Multiplication: " << num1 * num2 << endl;
13     cout << "Divison:" << num1 / num2 << endl;
14
15     Float num1("123456.7890");
16     Float num2("987.6543210");
17
18     cout << "Addition: " << num1 + num2 << endl;
19     cout << "Subtraction: " << num1 - num2 << endl;
20     cout << "Multiplication: " << num1 * num2 << endl;
21     cout << "Divison:" << num1 / num2 << endl;
22
23     return 0;
24
25 }
```

3 Limitations

The current implementation of the Infinite Arithmetic Library has the following limitations:

- **Error Handling:** The library does not handle special cases like NaN (Not a Number) or Infinity for floating-point arithmetic.
- **Performance Optimization:** The current implementation focuses on correctness and does not prioritise performance optimization techniques like parallelization.

4 Verification Approach:

- The Infinite Arithmetic Library was verified through a combination of unit testing and manual testing. Unit tests were written to cover various scenarios and edge cases for each arithmetic operation. These tests ensured that the library produces the expected results for a wide range of inputs.

- Manual testing was also performed by running the library with different sets of input values and verifying the correctness of the outputs.

5 Key Learnings:

Throughout the development of the Infinite Arithmetic Library, several key learnings emerged:

- **Testing and Verification:** Comprehensive testing and verification strategies were essential to ensure the correctness and reliability of the library, especially when dealing with large numbers and edge cases.
- **Code Organization:** Separating concerns into different classes and helper functions improved code readability, maintainability, and testability.

6 Git Commits SnapShot:

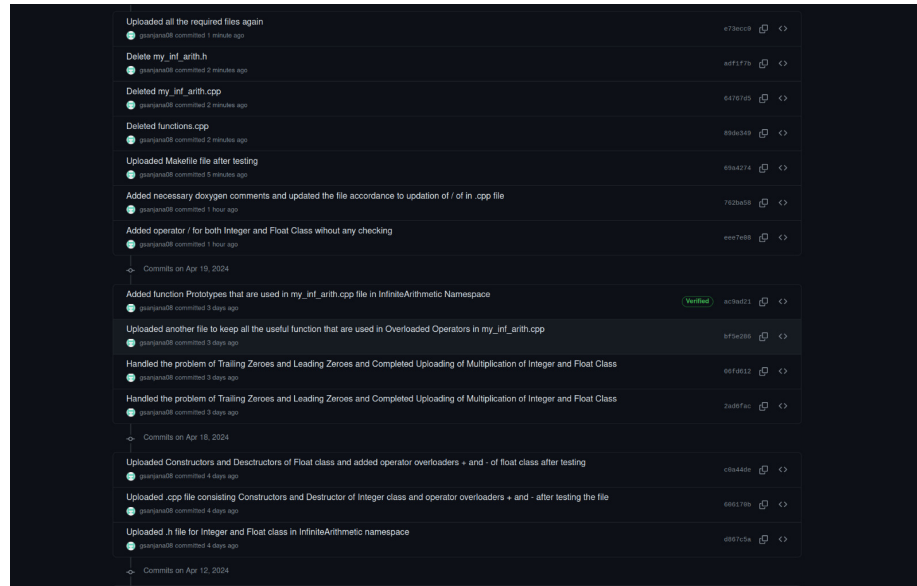


Figure 2: Snapshot of Git Commits